

1. AI Technique Used (Backtracking Search Algorithm)

This code demonstrates the **Backtracking Search AI** concept.

Backtracking Search for Doctor Shift Scheduling

doctors = ["D1", "D2", "D3"]

shifts = ["Morning", "Afternoon", "Night"]

def is_valid(assignment):

D1 cannot work Night

if assignment.get("D1") == "Night":

return False

D3 cannot work Morning

if assignment.get("D3") == "Morning":

return False

D2 must work before D3

if "D2" in assignment and "D3" in assignment:

if shifts.index(assignment["D2"]) >= shifts.index(assignment["D3"]):

return False

return True

def backtrack(assignment, doctors):

if len(assignment) == len(doctors):

return assignment

doctor = doctors[len(assignment)]

```
for shift in shifts:
```

```
    if shift not in assignment.values():
```

```
        assignment[doctor] = shift
```

```
    if is_valid(assignment):
```

```
        result = backtrack(assignment, doctors)
```

```
        if result:
```

```
            return result
```

```
    del assignment[doctor]
```

```
return None
```

```
solution = backtrack({}, doctors)
```

```
print("Valid Schedule:")
```

```
for doctor, shift in solution.items():
```

```
    print(doctor, "->", shift)
```

1. Problem Solution Code

This code directly solves the hospital scheduling problem.

```
# Hospital Doctor Shift Scheduling
```

```
schedule = {
```

```
    "D1": "Morning",
```

```
    "D2": "Afternoon",
```

```
    "D3": "Night"
```

```
}
```

```
print("Final Doctor Schedule")
```

```
print("-----")
```

```
for doctor, shift in schedule.items():
```

```
    print(doctor, "->", shift)
```

```
print("\nConstraint Verification")
```

```
print("D1 is not assigned Night Shift")
```

```
print("D2 works before D3")
```

```
print("D3 is not assigned Morning Shift")
```

```
print("One Doctor per Shift")
```

```
print("All Doctors Assigned")
```

```
print("\nSchedule is Valid")
```

2.AI Technique Used (Breadth First Search - BFS)

```
from collections import deque
```

```
graph = {  
    'S': ['A', 'B'],  
    'A': ['C'],  
    'B': ['D'],  
    'C': ['G'],  
    'D': [],  
    'G': []  
}
```

```
def bfs(graph, start, goal):
```

```
    queue = deque([[start]])
```

```
    visited = set()
```

```
    while queue:
```

```
        path = queue.popleft()
```

```
        node = path[-1]
```

```
        if node == goal:
```

```
            return path
```

```
        if node not in visited:
```

```
            visited.add(node)
```

```
            for neighbor in graph[node]:
```

```
new_path = list(path)
new_path.append(neighbor)
queue.append(new_path)
```

```
return None
```

```
path = bfs(graph, 'S', 'G')
```

```
print("Path Found:")
```

```
print(" -> ".join(path))
```

2. Problem Solution Code (Robot Navigation in 5×5 Grid)

```
from collections import deque
```

```
grid = [  
    ['S', 'O', 'O', 'O', 'O'],  
    ['O', 'X', 'X', 'O', 'O'],  
    ['O', 'O', 'O', 'X', 'O'],  
    ['O', 'X', 'O', 'O', 'O'],  
    ['O', 'O', 'O', 'O', 'G']  
]
```

```
rows = len(grid)  
cols = len(grid[0])
```

```
start = (0, 0)  
goal = (4, 4)
```

```
directions = [  
    (-1, 0),  
    (1, 0),  
    (0, -1),  
    (0, 1)  
]
```

```
queue = deque([(start, [start])])  
visited = set()
```

while queue:

 (x, y), path = queue.popleft()

 if (x, y) == goal:

 print("Shortest Path:")

 print(path)

 print("Path Cost =", len(path) - 1)

 break

 if (x, y) not in visited:

 visited.add((x, y))

 for dx, dy in directions:

 nx = x + dx

 ny = y + dy

 if (0 <= nx < rows and

 0 <= ny < cols and

 grid[nx][ny] != 'X' and

 (nx, ny) not in visited):

 queue.append(((nx, ny), path + [(nx, ny)]))

3. AI Technique Used (Uniform Cost Search - UCS)

```
from queue import PriorityQueue

graph = {
    'S': [('A', 2), ('B', 5)],
    'A': [('C', 3)],
    'B': [('D', 2)],
    'C': [('G', 4)],
    'D': [('G', 6)],
    'G': []
}

def uniform_cost_search(graph, start, goal):
    pq = PriorityQueue()
    pq.put((0, start, [start]))

    visited = set()

    while not pq.empty():
        cost, node, path = pq.get()

        if node == goal:
            return cost, path

        if node not in visited:
            visited.add(node)
```

```
        for neighbor, edge_cost in graph[node]:
            pq.put((cost + edge_cost,
                    neighbor,
                    path + [neighbor]))

    return None

cost, path = uniform_cost_search(graph, 'S', 'G')

print("Path Found:")
print(" -> ".join(path))
print("Total Cost =", cost)
```

3. Problem Solution Code (Autonomous Rescue Robot)

```
from queue import PriorityQueue
```

```
grid = [  
    ['S', 'O', 'R', 'O', 'O'],  
    ['O', 'X', 'X', 'O', 'O'],  
    ['O', 'O', 'R', 'X', 'O'],  
    ['O', 'X', 'O', 'O', 'O'],  
    ['O', 'O', 'O', 'O', 'G']  
]
```

```
rows = len(grid)  
cols = len(grid[0])
```

```
start = (0, 0)  
goal = (4, 4)
```

```
directions = [  
    (-1, 0),  
    (1, 0),  
    (0, -1),  
    (0, 1)  
]
```

```
pq = PriorityQueue()  
pq.put((0, start, [start]))
```

```
visited = set()

while not pq.empty():
    cost, (x, y), path = pq.get()

    if (x, y) == goal:
        print("Rescue Path Found:")
        print(path)
        print("Total Cost =", cost)
        break

    if (x, y) not in visited:
        visited.add((x, y))

        for dx, dy in directions:
            nx = x + dx
            ny = y + dy

            if (0 <= nx < rows and
                0 <= ny < cols and
                grid[nx][ny] != 'X' and
                (nx, ny) not in visited):

                move_cost = 1

                if grid[nx][ny] == 'R':
                    move_cost += 2

                pq.put((
```

```
cost + move_cost,  
(nx, ny),  
path + [(nx, ny)]  
)
```
